

Hack the North 2026: Build an AI Underwriting Agent

Welcome

You're building a real tool that insurance underwriters use every day: an AI agent that thinks about which submissions matter most.

The Challenge

Build an AI agent that thinks like an underwriter.

Given a queue of insurance submissions, your agent should: 1. Score each submission against carrier appetite guidelines 2. Reason about which data to request from the API 3. Rank submissions to surface the best opportunities 4. Explain every decision in plain English

This mirrors the work Federato's Forward Deployed Engineering (FDE) team does daily.

Why This Matters

Every day, underwriters review hundreds of insurance submissions. Each one needs checking against: - The carrier's appetite guidelines (which risks do we want?) - Outside risk data (flood zones, climate patterns, business health) - Portfolio context (are we already exposed to this risk?)

Today, this is manual, slow, and repetitive. A smarter agent could triage the queue in seconds and surface only the best matches.

What You're Given

1. Synthetic Dataset

A collection of 50+ realistic commercial insurance submissions. Access via a mock API.

2. Two API Endpoints

Schema Discovery:

```
POST https://product.federato.ai/integrations-api/handlers/federato-hack-north?outputOnly=true
{"action": "schema"}
```

Returns the structure of available data: resources, fields, types, and relationships.

Query Endpoint:

```
POST https://product.federato.ai/integrations-api/handlers/federato-hack-north?outputOnly=true
{"action": "query", "payload": {...}}
```

Runs queries against your data copy. Supports filtering, sorting, pagination, and aggregations.

3. Sample Appetite Guidelines

Real underwriting criteria from a 2025 carrier: preferred states, premium ranges, TIV (Total Insured Value) thresholds, building age/construction requirements, loss history limits.

4. Underwriting Glossary

Insurance and RiskOps terminology so you can speak the language.

5. Optional: External Data Enrichment

If you want to explore enriching submissions with outside data, you can research and integrate external APIs independently. This is completely optional and a bonus, not required.

3 examples (you'd research independently): - Nominatim (free geocoding via OpenStreetMap) - OpenFEMA API (free flood/disaster risk data) - Open-Meteo (free weather/climate history)

Key Milestones

Here's what successful solutions typically include. You might approach these in any order, emphasize different parts, or structure them completely differently—use this as guidance, not a prescription.

Understand the Domain

What: Get familiar with insurance terminology, what appetite guidelines are, and why they matter.

Guidance: - Read the Glossary to understand core concepts (Policy, Premium, Underwriting, Appetite, etc.) - Read the Appetite Guidelines Reference to see what factors real underwriters care about - Skim the Query Request Body to get a feel for the query language

Consider: - What makes a submission “good” vs. “bad”? - How would a human underwriter prioritize this queue? - What data would you need to make confident decisions?

Discover the Data Shape

What: Call the schema endpoint to understand what data is available. Figure out the structure without getting handed a field-by-field reference.

Guidance: - Get your `client_id` and `client_secret` from organizers - Follow the API Documentation to mint a token (**KEY:** Use `auth.product.federato.ai`, NOT the canonical domain) - Call the schema endpoint and explore the response - Save the schema—you'll reference it often

Consider: - What resources exist? - What fields are available on each resource? - Which fields are references to other resources? - What data structure aligns with the appetite guidelines?

Build Query Capability

What: Write and test queries against the API. Start simple, iterate.

Guidance: - Pick a simple goal (e.g., “find all active policies in California”) - Write a query against the schema you discovered - Test it. Does it return results? Are the results what you expected? - Iterate: add filters, refine projections, test pagination - Reference the Query Request Body guide for patterns and examples

Consider: - Can you filter by status, state, or business type? - Can you follow references to get related data (e.g., broker info)? - Can you use aggregations to summarize (e.g., count policies, sum TIV)? - What queries will you need for the next milestone?

Build Agentic Reasoning

What: Create a layer that reasons about which queries to construct, rather than hardcoding them.

Guidance: - Don’t hardcode every query by hand. That’s manual and doesn’t scale. - Build something that can translate a goal into a query—could be templating, rule-based logic, or an LLM prompt - Have your agent inspect the schema to understand available fields - Have it reason about which queries would help answer the question - Let it execute queries and adapt if needed (e.g., if few results, try a broader filter)

Consider: - Can your agent understand “find policies that match our appetite”? - Can it translate that into the right queries? - Does it understand tradeoffs (e.g., when to expand references vs. when not to)? - Can it explain its reasoning (either to you or to the underwriter)?

This is where you differentiate. Simple template-based logic is fine. Full LLM integration is fine too. The point is: does your agent think?

Score and Rank Submissions

What: Apply appetite guidelines to score each submission and rank the queue.

Guidance: - Implement appetite-matching logic: does each submission fit the guidelines? - Decide how to calculate a score. Could be: - Simple: +points for each matched criterion, -points for mismatches - Weighted: different criteria matter different amounts - Rules-based: “if X and Y, then high priority” - LLM-based: ask a model to score given the guidelines - Rank submissions from highest to lowest score - Decide on your top-N list (e.g., top 10 or top 20)

Consider: - How do you handle contradictions (e.g., in-appetite on some factors, out-of-appetite on others)? - Do all appetite criteria matter equally, or do some matter more? - How transparent is your scoring?

Add Decision Explanations

What: For every submission in your output, explain why it got that score.

Guidance: - This is critical. An underwriter needs to understand *why* a submission is ranked high or low. - For each result, add a 2-3 sentence explanation covering: - How it matches (or doesn’t match) appetite guidelines - Key decision factors - A recommendation (review, accept, reject, investigate further)

Example:

```
Policy #42: SCORE 87/100
$45M TIV commercial property in CA matches your appetite targets.
Building is newer than 2010, premium $85K in range, no claims in 5 years.
Recommendation: Review for acceptance.
```

Consider: - Can a real underwriter read your explanation and understand your logic? - Are you transparent when a submission is borderline or contradictory? - Does your explanation help them take action (not just show a number)?

Build a UI or Present Results

What: Show your ranked submissions and explanations in a usable format.

Guidance: - Could be a web dashboard, a CLI tool, a printed report, or an API endpoint - Include: - Ranked list (high-to-low score) - Per-submission breakdown (key factors, scores) - Explanations (why it got that score) - Optional: filter/sort options, visualizations

Consider: - Clarity matters more than polish. A simple table with good explanations beats a fancy interface hiding unclear logic. - Can an underwriter quickly understand what they're looking at? - Can they take action (review, approve, reject)?

Optional: Enrich with External Data

What: Research and integrate external APIs to add real-world risk signals.

Guidance: - This is completely optional. You can score highly without it. - Pick 1-2 APIs that make sense for underwriting (e.g., geocoding, flood risk, weather history) - Research their documentation and understand authentication (many don't require API keys) - Integrate them into your solution - Make sure enriched data actually affects your ranking (not just for show) - Explain how the enrichment changed your decisions

Consider: - Which external data would a real underwriter care about? - Which 3 example APIs would be most valuable? (Nominatim, OpenFEMA, Open-Meteo are just suggestions) - Is enrichment worth the time investment right now, or would you rather focus on core reasoning?

What “Good” Looks Like

Minimum Viable Solution

- Queries the API successfully
- Applies appetite guidelines using logic (rules, templates, LLM)
- Ranks submissions by a calculated score
- Shows a list with brief explanations
- Handles 50+ submissions in reasonable time

Strong Solution

- All of MVP, plus:
- Your agent constructs queries dynamically (not hardcoded every submission)
- Explanations are detailed and justify every decision
- Handles edge cases (missing fields, API issues)
- Clean code, good error messages

Exceptional Solution

- All of Strong, plus:
- Clear agentic reasoning (you can trace *why* it chose specific queries)
- Adapts based on results (e.g., deepens analysis for high-value opportunities)
- Explanations address contradictions transparently
- Polished UI/output that makes insights immediately actionable

With Bonus Enrichment

- Any of above, plus:
 - 1-2 thoughtfully researched external APIs
 - Enrichment data visibly influences ranking
 - Clear explanation of how enrichment changed decisions
-

Common Pitfalls & How to Avoid Them

1. Hardcoding Queries

Problem: You write every query by hand. **Why it's bad:** Manual and doesn't scale. Misses the agentic reasoning aspect. **Solution:** Build a templating or rule-based layer that translates appetite rules into queries.

2. Ignoring Schema Discovery

Problem: You look at sample data once, then assume field names. **Why it's bad:** Queries break if you misread the schema. **Solution:** Call the schema endpoint at startup. Let your agent learn the structure.

3. No Explanations

Problem: You output a ranked list with scores, but no reasoning. **Why it's bad:** Underwriters can't confidently act on the results. **Solution:** For each result, add a 2-3 sentence explanation: appetite match + key factors + recommendation.

4. Dot-Paths on Arrays

Problem: You try `{"locations.state": "CA"}` to find submissions from CA. **Why it's bad:** Dot-paths don't traverse arrays. Silently matches nothing. **Solution:** Use `$elemMatch`: `{"locations": {"$elemMatch": {"state": "CA"}}}`

5. References Without \$expand

Problem: You select a nested field like `producer.broker.name` without expanding the reference first.

Why it's bad: You get back IDs (like `2`), not the actual broker name.

Solution: Use `$expand` in your select clause to hydrate references:

```
{ "select": { "producer": { "broker": { "$expand": { "select": ["name"] } } } } }
```

6. Overthinking Enrichment

Problem: You spend most of your time researching and integrating external APIs. **Why it's bad:** The core challenge is agentic reasoning, not API integration. **Solution:** Build your core agent first. Enrichment is optional bonus if you have time.

7. UI Over Reasoning

Problem: You spend 90% of time on UI, 10% on core logic. **Why it's bad:** Weak reasoning with a pretty interface doesn't help underwriters. **Solution:** Prioritize: (1) agentic reasoning, (2) explanations, (3) UI polish.

FAQ

Setup & Auth

Q: Auth keeps failing with “401 Invalid token.” A: You're minting from the wrong Auth0 domain. Use `auth.product.federato.ai`, NOT the canonical domain. See the API Documentation for the exact endpoint.

Q: How long do tokens last? A: 4 hours. If running a long session, mint a fresh token.

Data & Queries

Q: Why does my query with `locations.state = "CA"` return nothing? A: Dot-paths don't traverse arrays. Use `$elemMatch` instead: `{"locations": {"$elemMatch": {"state": "CA"}}}`

Q: How do I follow a reference (e.g., get the broker name)? A: Use `$expand` in your select. Example: `{"producer": [{"broker": {"$expand": {"select": ["name", "license_number"]}}}]}`

Q: What's the difference between `where` and `filter`? A: `where` runs before references are expanded (pre-filter). `filter` runs after expansion. Use `where` for whole-submission filters, `filter` for post-expansion filters.

Enrichment

Q: Do I have to use external APIs? A: No. External enrichment is completely optional. You can score highly without it by focusing on agentic reasoning and clear explanations.

Q: Can I use a different external API? A: Yes, absolutely. The 3 examples are just suggestions.

Q: Will not using enrichment hurt my score? A: No. The core challenge is reasoning + explainability.

General

Q: Can I hardcode the queries? A: Technically yes, but it's risky and less interesting. A small templating layer is worth the effort.

Q: Can I use Claude, ChatGPT, or another LLM? A: Yes. No restrictions on tools.

Q: Do I have to build a web UI? A: No. A CLI, dashboard, API, or even printed output is fine. Clarity matters more than polish.

Q: What if I'm stuck? A: Check the API Documentation and Query Request Body guide. Ask organizers for help. The technical docs are detailed and have worked examples.

Q: What if an external API goes down (if I'm using one)? A: Handle it gracefully. Log the error, proceed without that enrichment, or skip that submission.

Q: Can I approach these milestones in a different order? A: Absolutely. This is one way to think about it. You might understand domain + discover data together, or score + explain together. Build in the order that makes sense to you.

Resources & Links

All of these are available in the Hack the North 2026 Notion workspace:

1. **API Documentation** - Auth flow, endpoints, troubleshooting, curl examples
 2. **Query Request Body** - Complete query language reference with worked examples
 3. **Data Schema** - Explanation of how schema discovery works
 4. **Glossary** - Insurance and RiskOps terminology
 5. **Appetite Guidelines Reference** - Real underwriting criteria for the sample carrier
-

Key Reminders

Focus on reasoning. The core challenge is building an agent that thinks about what data to request and why. Teams that demonstrate clear thinking will stand out.

Explain decisions. An underwriter needs to understand *why* a submission is ranked high or low. Clear explanations are critical.

External APIs are optional. Enrichment is a bonus, not required. Don't spend your entire hackathon on external integrations.

You control the structure. These milestones are one way to organize the work. You might tackle them in a different order, spend more time on some and less on others, or combine them differently. Build in the way that makes sense to you.

Test early. Get one successful query working quickly, then iterate. Don't get stuck on perfect schema understanding upfront.

Keep it simple. Clean code, clear explanations, and working end-to-end beats fancy tech or UI polish.

Good Luck!

You're building a real piece of the insurance underwriting workflow. Teams that nail the thinking part-that can reason about what data to request and explain why-will excel.

Questions? Check the technical docs or ask an organizer.

Now go build an underwriting agent!